SOLUTION

PART A **[70 MARKS]**

QUESTION 1 **[4 Marks]**

```
int hours[no_cars];          //1M
double rate[no_cars];        //1M
hours[index];                //0.5M
rate[index];                 //0.5M
totalPay = hours[index] * rate[index]; //1M
```

QUESTION 2 **[16 Marks]**

(a) (2 marks)

```
double sugarLevel[4][3]  = { {4.5, 5.7, 6.8},
                             {6.2, 7.1, 7.3},
                             {5.1, 6.3, 8.0},
                             {7.5, 8.5, 7.3}};
```

//0.25M for each item

//0.25M for each line

(b) (3 marks)

```
cout<<" Age of Patients" <<endl;     //0.5M
for (int i=0; i<4; i++)              //1M
{
        cout << patients[i]
             << "\t" << age[i]
             << endl;                //1.5M
}
```

(c) (4 marks)

```
cout << "Name"<< "\t"<<"Breakfast"<< "\t" << "Lunch"
     << "\t" <<"Dinner"<<endl;            //0.5M

for (int i=0; i<4; i++)                   //1M
{
```

```
        cout <<patients[i] << "\t";      //0.5M
        for (int j=0; j<3; j++)          //0.5M
            cout << sugarLevel[i][j] << "\t"; //1M

        cout << endl;                    //0.5M
    }
```

(d) (7 marks)

```
    cout<<" Sugar Level of Patients" <<endl;        //0.5M
    for (int i=0; i<4; i++)                         //1M
    {
        total = 0;                                  //0.5M

        //Sum row
        for (int j=0; j<3; j++)                     //1M
            total += sugarLevel[i][j];              //1.5M

        //Get the average
        averageSugarLevel[i] = total/ 3;            //1M

        //Display the average
        cout << patients[i]
            << "\t" << averageSugarLevel[i]
            << endl;                                //1.5M
    }
```

QUESTION 3 **[11 Marks]**

(a) (2 marks)

```
open(), close(), fail() //2M @ (any 2 answer)
```

(b) (9 marks)

```
fin.open("file1.dat"); //1M
if (!fin) //1M
{
    cout <<"Input file opening failed.\n"; //0.5M
    return 0; //1M @ exit(1);
}
fout.open("file2.dat"); //1M
int num; //0.5M
for (int i = 0; i < 10; i++) //1M
{
    fin >> num;  //1M
    fout << num; //1M
}
```

```
fin.close(); //0.5M
fout.close();//0.5M
```

QUESTION 4 **[4 Marks]**

Table 1: Output file named **`Jadual.txt`**

1	5	0	1	.	0				3	.	6		9	8	4	.	0
			5	.	3			9	1	.	0	1	9	7	9	.	0
		2	4	.	0		6	0	6	.	0			1	1	.	0

0.5M (for each value) × 6 = 3M

Right indentation = 0.5M

Showpoint = 0.5M

QUESTION 5 **[4 Marks]**

```
//function prototype
void convert (double *);    //1M

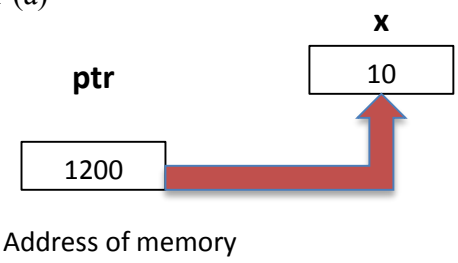
//function convert
void convert (double *input) //1M
{
    *input = ((*input - 32) * 5) / 9; //2M
}
```

QUESTION 6 **[10 Marks]**

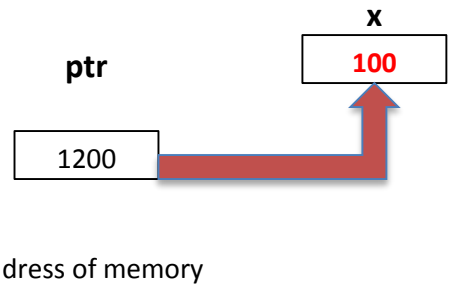
(a) (7 marks)

```
10   20    30
100  80    60
//0.5M each value
//Total = 3 marks
```

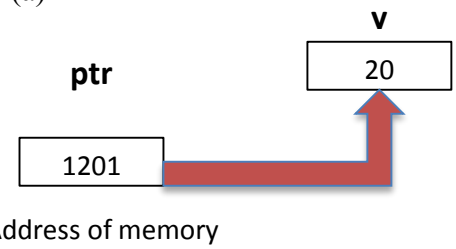
Step 1 (a)



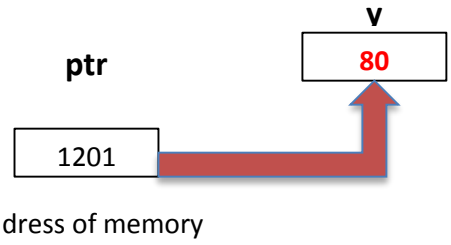
Step 1 (b)



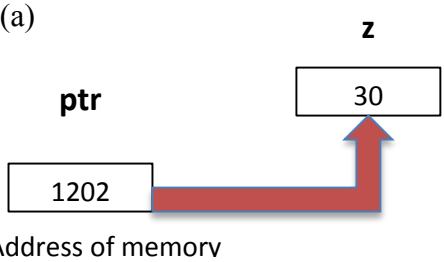
Step 2 (a)



Step 2 (b)



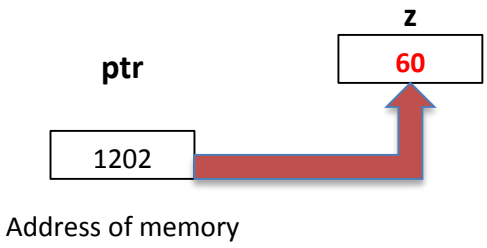
Step 3 (a)



Notes:

1. *Address of memory can be any.*
2. *For each step, marks given for correct variable, value, pointer, updated value. (0.25M each × 4 = 1M)*
3. *Each step 1M (3 steps × 1M=3M)*
4. *Show the address and arrow (1M)*
5. *Total = 4 marks*

Step 3 (b)



(b) (3 marks)

```
8
2
2    4
4    2
```

//0.5M each value

//Total = 3 marks

QUESTION 7 **[5 Marks]**

- The semicolon is missing after the closing brace.
- Structure members cannot be initialized in the structure definition.
- The statement `point.x[0] = 1` should read as `point[0].x = 1;`
- The statement `point.y[0] = 1` should read as `point[0].y = 1;`
- An entire `struct` cannot be sent to `cout`.

1M (for each error) × 5 = 5M

QUESTION 8 **[16 Marks]**

(a) (6 marks)

```
//(i) 2 marks
struct TempScale //1M
 {
   double fahrenheit;  //0.5M
   double celsius; //0.5M
 };
```

```
//(ii) 3 marks
struct Reading  //1M
{
  int windSpeed;   //0.5M
  double humidity; //0.5M
  tempScale temperature;  //1M
};

//(iii) 1 mark
Reading today; //1M
```

(b) (3 marks)

```
today.windSpeed = 8;      //0.5M
today.humidity = 0.53;    //0.5M
today.temperature.fahrenheit = 80.2; //1M
today.temperature.celsius = 26.8;    //1M
```

(c) (7 marks)

```
void showReading (Reading values) //2M
{
   cout << "Windspeed: " << values.windSpeed << endl; //1M
   cout << "Humidity: " << values.humidity << endl; //1M
   cout << "Fahrenheit: " << values.temperature.fahrenheit
        << endl; //1.5M
   cout << "Celsius: " << values.temperature.celsius
        << endl; //1.5M
}
```